

Reusable Assets

A collection of developer tools built with Angular and the H5 SDK for Infor M3 Cloud.

[Get Started]({{ site.baseurl }}{% link docs/getting-started/getting-started.md %}) [View on GitHub](#)

Overview

Reusable Assets is an Angular 18 application that provides productivity tools for M3 developers and consultants. It integrates with M3 APIs, ION, Event Hub, and XtendM3 to streamline common development workflows.

Key Features

Feature	Description
XtendM3 CRUD Generator	Generate full CRUD transactions for custom tables and upload directly to M3
Event Hub Formatter	Browse, filter, and export Event Hub subscriptions
Custom List Checker	Validate custom list configurations from ZAP files or MEC mappings
MEC Mapping Viewer	Browse MEC mappings with Java source code viewer
Dataflow List	Extract ION dataflow dependencies and generate DES-030 documents
Object Schema Editor	Browse and edit ION noun XSD definitions
MEC Variable Validations	Generate null-safe trim statements from MEC Java classes
GenAI Chat	AI chat interface powered by Infor's GenAI service
Kiro AI Chat	AI-powered chat assistant with WSL backend
Generators	Quick timestamp and UUID generators

Author

Maintained by **Romeo Bardillon**

Tech Stack

- **Framework:** Angular 18.2 with TypeScript
- **UI Library:** IDS Enterprise (Soho) via `ids-enterprise-ng`
- **M3 Integration:** `@infor-up/m3-odin` and `@infor-up/m3-odin-angular`
- **Code Editor:** Monaco Editor
- **Build Tool:** Angular CLI

Getting Started

Set up your development environment and run the application.

Prerequisites

Table of contents

Required Software

Software	Version	Purpose
Node.js	18+	Runtime
Angular CLI	18.2+	Build & serve
npm	9+	Package management

M3 Environment

The application requires access to an Infor M3 Cloud Edition environment with:

- ION API Gateway access
- M3 API (MI) endpoints
- Event Hub subscriptions (for Event Hub Formatter)
- XtendM3 Extensibility API (for CRUD Generator)

- Grid API (for MEC Mapping Viewer)

Optional: Kiro Chat Backend

The Kiro Chat tab requires a local backend running in WSL. See [Kiro Chat Setup](#) for details.

Installation

```
cd "reusable-assets"  
npm install
```

Running the Application

```
ng serve
```

Navigate to <http://localhost:4200>.

Running inside H5 SDK

When deployed as an H5 SDK widget, the application runs within the M3 context and automatically inherits authentication from the user session.

Features

Detailed documentation for each tool available in Reusable Assets.

XtendM3 CRUD Generator

Generate full CRUD transactions for XtendM3 custom tables and upload them directly to M3.

Table of contents

Overview

The XtendM3 CRUD Generator creates complete Add, Delete, Get, List, and Update transactions for custom dynamic tables. It handles code generation, direct upload to the M3 Extensibility API, and table definition management.

Capabilities

Transaction Generation

- Generates **Add**, **Del**, **Get**, **Lst**, and **Upd** transactions
- Configurable field definitions (name, type, length, mandatory, key)
- Auto-generates proper XtendM3 Groovy code with database operations

Upload to M3

- Upload transactions directly to the M3 Extensibility API
- Auto-activation after upload
- Upload table definitions to Foundation REST `importTables` endpoint (with CSRF token handling)

Configuration Management

- Save/load field configurations to M3 environment via `EXT999MI`
- Download generated code as ZIP archive
- Export/import JSON configurations for sharing

User Context

- User field auto-populated from M3 login
- Read-only when authenticated within M3 session

Usage

1. Define your table fields (name, type, length, key fields)
2. Click **Generate** to preview the CRUD transactions
3. Use **Upload** to push directly to M3, or **Download ZIP** for offline use
4. Optionally save your configuration for reuse via **Save Config**

Event Hub Formatter

Browse, filter, and export Event Hub subscription events from your M3 environment.

Table of contents

Overview

The Event Hub Formatter connects to your M3 environment's Event Hub to browse subscriptions, query events with filters, and export results.

Capabilities

Subscription Browsing

- Browse Event Hub subscriptions directly from the connected M3 environment
- Multi-select subscriptions for batch querying

Filtering

- Start time and time span configuration
- Criteria-based filtering on event properties

Results Display

- Filterable and sortable datagrid for event results
- Row selection opens event detail in a Soho modal dialog
- Flattened element columns for easy reading

Export

- Export events to CSV with flattened element columns
- Existing CSV/text formatting preserved for compatibility

Usage

1. Select one or more subscriptions from the dropdown
2. Configure start time, time span, and optional criteria
3. Click **Search** to retrieve events
4. Click any row to view full event detail
5. Use **Export CSV** to download results

Custom List Checker

Validate custom list configurations from ZAP files or MEC mappings.

Table of contents

Overview

The Custom List Checker validates that CMS100MI/MDBREADMI transactions referenced in custom lists are properly configured. It supports both ZAP file upload and direct MEC mapping selection.

Capabilities

ZAP File Upload

- Upload ZAP configuration files
- Automatically extracts CMS100MI/MDBREADMI transaction references
- Displays results in a filterable/sortable datagrid

MEC Mapping Dropdown

- Fetched from Grid API (`/grid/appui/EC_MT`)
- Select a mapping to auto-extract transaction references
- Loading state indicator while fetching mappings
- Alternative to uploading ZAP files — same datagrid output

Usage

Option A — ZAP File:

1. Click **Upload** and select a ZAP file
2. Review extracted transactions in the datagrid

Option B — MEC Mapping:

1. Select a mapping from the dropdown
2. Transactions are automatically extracted and displayed

MEC Mapping Viewer

Browse MEC mappings and view compiled Java source code with syntax highlighting.

Table of contents

Overview

The MEC Mapping Viewer provides a read-only interface to browse all MEC (M3 Enterprise Collaborator) mappings in your environment and inspect their compiled Java source code.

Capabilities

Mapping Browser

- Filterable and sortable datagrid of all MEC mappings
- Shared Grid API service — mappings loaded once, cached across tabs
- Supports both admin and read-only MEC environments

Code Viewer

- Monaco Editor integration for Java source code display
- Syntax highlighting, minimap, and search
- Read-only view of compiled mapping logic

Usage

1. Browse mappings in the datagrid
2. Select a mapping row to load its Java source
3. Use Monaco's built-in search (**Ctrl+F**) to navigate the code

Kiro AI Chat

AI-powered chat assistant with a local WSL backend.

Table of contents

Overview

The Kiro Chat tab provides an AI chat interface powered by the Kiro CLI. It requires a local Node.js backend running in WSL that spawns `kiro-cli` and streams responses back via Server-Sent Events (SSE).

Architecture

```
Angular App (localhost:4200)
└─ proxy /kiro-chat/* ─► Node.js backend (localhost:3000)
                          └─ spawns kiro-cli in WSL
                          └─ streams SSE responses
```

Setup

1. Install WSL

```
wsl --install
```

2. Install Node.js 20+ in WSL

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs
```

3. Install and authenticate Kiro CLI

```
npm install -g kiro-cli
kiro-cli login
```

4. Start the backend

Double-click `start.bat` in the project root, or manually:


```
wsl
cd /mnt/c/Users/<your-user>/Coding/kiro-chat-server
npm install
node server.js
```

5. Start the Angular app

```
ng serve
```

Both servers must be running simultaneously for the chat to work.

Proxy Configuration

The proxy is configured in `odin.json` :

```
{
  "/kiro-chat": {
    "target": "http://localhost:3000",
    "secure": false,
    "changeOrigin": true,
    "pathRewrite": { "^/kiro-chat": "" }
  }
}
```

Dataflow List

Browse ION dataflows, extract dependencies, and generate DES-030 documents.

Table of contents

Overview

The Dataflow List tab connects to the ION Connect API to list all dataflows in your environment. Selecting a dataflow extracts its full dependency tree — mappings, connection points, file templates, object schemas, scripts, workflows, and custom lists — and can generate a DES-030 design document.

Capabilities

Dependency Extraction

- ION Mappings
- Connection Points (with file template resolution)
- Application Connections
- Splitters & CBR Filters
- Scripts & Workflows
- Object Schemas (nouns)

Custom List Resolution

- Automatically matches APPLICATION nouns to MEC mappings
- Extracts CMS100MI/MDBREADMI transactions from matched mappings
- Fetches custom list data via `EXT100MI.LstCustListData`

DES-030 Generation

- Generates a Word document (`.docx`) with all extracted dependencies
- Configurable user story, title, and change references
- Includes connection point details and custom list data

Clipboard Copy

- Copy all grouped dependency info to clipboard in text format

Usage

1. Select a dataflow from the dropdown
2. Review the extracted dependency groups
3. Optionally fill in DES-030 fields (user story, title, change refs)
4. Click **Generate DES-030** to download the Word document, or **Copy** to clipboard

GenAI Chat

AI chat interface powered by Infor's GenAI service.

Table of contents

Overview

The GenAI Chat tab provides a conversational AI interface that connects to Infor's GenAI backend. It supports session management, message history, and streaming responses.

Capabilities

Session Management

- Create new chat sessions
- Browse and resume previous sessions (sorted by last updated)
- View full message history for any session

Streaming Responses

- Real-time streaming of AI responses via SSE
- Human and AI messages displayed in a chat UI
- Enter to send, Shift+Enter for newline

Usage

1. Type a message and press **Enter** to start a new session
2. Click the sessions panel to browse or resume previous conversations
3. Click **New Chat** to start fresh

Generators

Quick utility generators for common development values.

Overview

The Generators tab provides simple value generators that are frequently needed during M3 development and testing.

Available Generators

Generator	Output	Format
Timestamp	Current date/time	YYYYMMDDHHmmssCC (centiseconds)
UUID	Random UUID v4	xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx

Usage

1. Select a generator from the dropdown
2. Click **Generate**
3. The result is automatically copied to your clipboard

MEC Variable Validations

Generate null-safe trim statements from MEC Java class fields.

Overview

The MEC Variable Validations tab takes a Java class (typically from a MEC mapping) and generates null-safe trim statements for all `String` fields. This is a common pattern needed in MEC mappings to sanitize input variables.

How It Works

Paste a Java class containing `private String` field declarations. The tool extracts all field names and generates:

```
fieldName = (fieldName == null) ? "" : fieldName.trim();
```

for each field.

Usage

1. Paste your MEC Java class into the input editor
 2. Click **Generate**
 3. Copy the generated trim statements from the output editor into your mapping
-

Object Schema Editor

Browse, edit, and update ION object schema (noun) XSD definitions.

Table of contents

Overview

The Object Schema Editor connects to the ION Data Catalog API to list available nouns, load their XSD schema content, and push updates back to the catalog.

Capabilities

Noun Browsing

- Lists all nouns from the ION Data Catalog
- Filters by tenant (e.g. shows only tenant-specific nouns for Goldwin environments)
- Refresh button to reload the noun list

Schema Editing

- Monaco Editor with XML syntax highlighting
- Auto-resizing editor (up to 700px height)
- Built-in XML formatter for readability

Schema Update

- Push modified XSD back to the ION Data Catalog
- Validation feedback on success or failure
- User context auto-populated for audit trail

Usage

1. Select a noun from the dropdown
2. Click **Load** to fetch its XSD content
3. Optionally click **Format** to pretty-print the XML

4. Edit the schema as needed
5. Click **Update** to push changes to the Data Catalog

Architecture

Technical overview of the application structure and services.

Project Structure

Table of contents

Directory Layout

```
src/app/
├── components/
│   ├── main/                # Root layout component
│   └── tabs/                 # Feature tabs
│       ├── custom-bod-generator/
│       ├── custom-list-checker/
│       ├── dataflow-list/
│       ├── event-hub-formatter/
│       ├── genai-chat/
│       ├── generators/
│       ├── kiro-chat/
│       ├── mec-mapping-viewer/
│       ├── mec-variable-validations/
│       ├── object-schema-editor/
│       └── xtendm3-crud-generator/
└── generators/
```

```
|   └─ des030/                # DES030 code generators
|   └─ service/              # Shared services
|   └─ app.module.ts         # Root module
```

Key Dependencies

Package	Purpose
@infor-up/m3-odin	M3 API client and utilities
@infor-up/m3-odin-angular	Angular bindings for M3 Odin
ids-enterprise-ng	Infor Design System (Soho) components
monaco-editor	Code editor for MEC viewer and generators
jszip	ZIP file generation for downloads
fast-xml-parser	XML parsing for ZAP/BOD files
file-saver	Client-side file downloads
docx	Word document generation

Services

Shared Angular services that provide M3 connectivity and business logic.

Table of contents

MI API Service

`mi-api.service.ts`

Wraps `@infor-up/m3-odin` to call M3 API (MI) transactions. Used by most tabs for CRUD operations against standard and custom APIs.

Grid API Service

```
grid-api.service.ts
```

Connects to the M3 Grid API for fetching MEC mappings and other grid-based data. Results are cached across tabs to avoid redundant network calls.

ION Connect Service

```
ion-connect.service.ts
```

Handles ION API Gateway connectivity including OAuth token management and endpoint routing.

ION Data Catalog Service

```
ion-data-catalog.service.ts
```

Interfaces with the ION Data Catalog for metadata and schema operations.

User Context Service

```
user-context.service.ts
```

Provides the current M3 user context (company, division, user ID). Auto-populated from the M3 session when running inside H5.

Message Service

```
message.service.ts
```

Centralized notification/messaging service for displaying alerts and status messages across components.

Kiro Chat Service

```
kiro-chat.service.ts
```

Manages SSE connections to the local Kiro backend, handling message streaming and connection lifecycle.

GenAI Chat Service

```
genai-chat.service.ts
```

Service layer for the GenAI chat integration.

DES030 Generator Service

`des030-generator.service.ts`

Business logic for generating DES030 (Experience Designer) configurations.

Configuration

Environment and proxy configuration for development and deployment.

odin.json Configuration

Table of contents

Overview

The `odin.json` file configures the H5 SDK project name and proxy rules for local development. Proxy entries route API calls to the M3 Cloud environment during `ng serve`.

Structure

```
{
  "projectName": "reusable-assets",
  "proxy": {
    "/m3api-rest": {
      "target": "https://mingle-ionapi.
<region>.inforcloudsuite.com/<TENANT>",
      "secure": false,
      "changeOrigin": true
    },
  },
}
```

```

"/mne": {
  "target": "https://<mne-host>:8080",
  "secure": false,
  "changeOrigin": true
},
"/ca": {
  "target": "https://<ca-host>:8080",
  "secure": false,
  "changeOrigin": true
},
"/kiro-chat": {
  "target": "http://localhost:3000",
  "secure": false,
  "changeOrigin": true,
  "pathRewrite": { "^/kiro-chat": "" }
}
}
}

```

Proxy Entries

Path	Target	Purpose
/m3api-rest	ION API Gateway	M3 API (MI) calls
/mne	MNE server	MEC/Event Hub endpoints
/ca	CA server	Grid API / Data Catalog
/kiro-chat	localhost:3000	Local Kiro CLI backend

Deployment

When deployed as an H5 widget inside M3, proxy rules are not needed — the application runs within the same origin as M3 and inherits the session context.

Kiro Chat Setup

Table of contents

Prerequisites

- Windows with WSL installed
- Node.js 20+ inside WSL
- Kiro CLI installed and authenticated

Step-by-step

1. Install WSL

```
wsl --install
```

Restart if prompted.

2. Install Node.js in WSL

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -  
sudo apt-get install -y nodejs
```

3. Install Kiro CLI

```
npm install -g kiro-cli  
kiro-cli login
```

Follow the browser-based OAuth flow.

4. Start the backend

Use `start.bat` in the project root, or:

```
wsl  
cd /mnt/c/Users/<your-user>/Coding/kiro-chat-server  
npm install  
node server.js
```

The backend starts on `http://localhost:3000` .

5. Verify

With both `ng serve` and the backend running, navigate to the **Kiro Chat** tab at <http://localhost:4200>.

Both servers must be running simultaneously for the chat to function.
